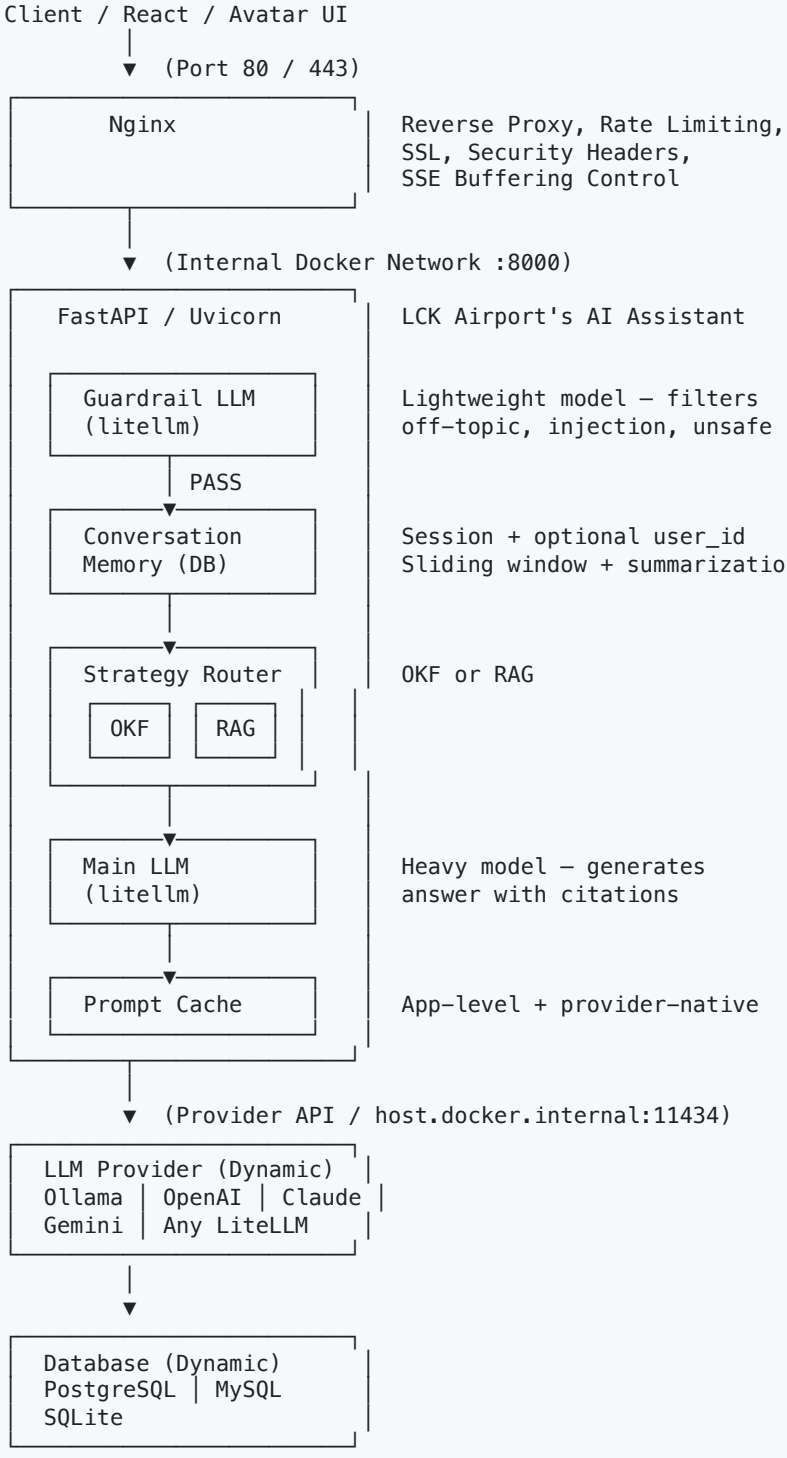
LCK Airport's AI Assistant — Backend Architecture

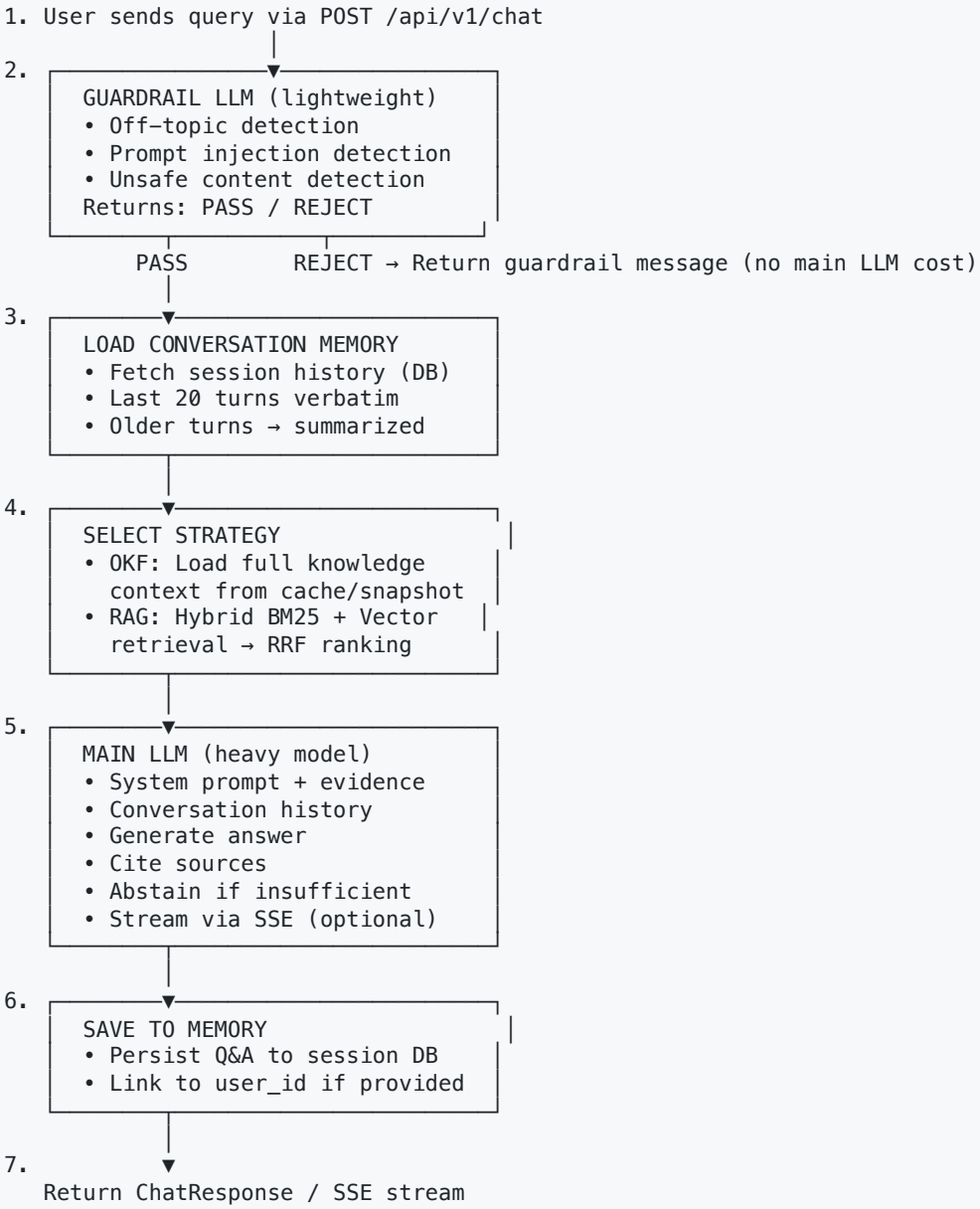
1. Project Identity

Field	Value
Project Name	LCK Airport's AI Assistant
Airport	Chaudhary Charan Singh International Airport (LCK), Lucknow
Backend Language	Python 3.12+ (FastAPI + Uvicorn)
LLM Integration	LiteLLM (unified interface for all providers)
Reverse Proxy	Nginx
Containerization	Docker + Docker Compose

2. Architecture



3. Request Lifecycle



4. Directory Structure

```
airport_project/
│
├── README.md
├── backend.md                          # This file
├── .gitignore
│
├── data/                               # Knowledge & evaluation data (separate from code)
│   ├── manifest.json                   # Approved knowledge document manifest
│   ├── lck_airport/                    # Approved Markdown knowledge documents
│   │   ├── terminal_facilities.md
│   │   ├── flight_and_parking.md
│   │   └── baggage_and_security.md
│   └── evaluation/                     # Benchmark & test datasets
│       ├── ground_truth.sample.json    # Small sanitized set (committed to Git)
│       └── unknown_traps.sample.json   # Adversarial traps sample (committed to Git)
│
└── backend/
    ├── compose.yaml                    # Docker Compose (Nginx + Backend + DB services)
    ├── pyproject.toml                  # Python dependencies & packaging
    ├── .env.example                    # Template env vars (committed to Git)
    ├── .env                            # Runtime env vars (gitignored)
    │
    ├── docker/                         # All container configurations
    │   ├── Dockerfile                  # FastAPI / Python container
    │   ├── .dockerignore
    │   └── nginx/
    │       ├── Dockerfile              # Nginx container
    │       ├── nginx.conf              # Global Nginx config (worker, gzip, timeouts)
    │       └── conf.d/
    │           └── default.conf        # Reverse proxy, SSE streaming, rate limits, headers
    │
    ├── src/
    │   └── lck_backend/
    │       ├── __init__.py
    │       ├── main.py                 # FastAPI app factory & lifespan
    │       │
    │       ├── api/                    # Route handlers
    │       │   ├── __init__.py
    │       │   ├── router.py           # Main v1 API router
    │       │   ├── dependencies.py     # Auth (Bearer token), service injection
    │       │   └── routes/
    │       │       ├── __init__.py
    │       │       ├── health.py       # GET  /api/v1/health
    │       │       ├── knowledge.py    # GET  /api/v1/knowledge/status
    │       │       │                   # POST /api/v1/knowledge/refresh (protected)
    │       │       └── chat.py         # POST /api/v1/chat (OKF/RAG, optional SSE)
    │       │
    │       ├── core/                   # Application settings & utilities
    │       │   ├── __init__.py
    │       │   ├── config.py           # Pydantic BaseSettings (all dynamic env vars)
    │       │   ├── logging.py          # Structured logging setup
    │       │   └── errors.py           # Custom exceptions & error handlers
    │       │
    │       ├── schemas/                # Pydantic request/response models
    │       │   ├── __init__.py
    │       │   ├── chat.py             # ChatRequest, ChatResponse, Citation, StreamChunk
    │       │   ├── knowledge.py        # Manifest, DocumentMetadata, SnapshotStatus
    │       │   └── health.py           # HealthResponse, ServiceStatus
    │       │
    │       ├── llm/                    # LiteLLM integration
    │       │   ├── __init__.py
    │       │   └── client.py           # Unified completion & streaming via litellm
    │       │
    │       ├── guardrail/              # Query safety gate
    │       │   ├── __init__.py
    │       │   ├── classifier.py       # PASS / OFF_TOPIC / UNSAFE / INJECTION classification
    │       │   └── prompts.py          # Guardrail system prompt templates
```

```
│   │   │
│   │   ├── memory/                    # Conversation memory management
│   │   │   ├── __init__.py
│   │   │   ├── base.py                # MemoryStore ABC
│   │   │   ├── factory.py             # Dynamic DB backend factory
│   │   │   ├── models.py              # SQLAlchemy ORM models (Session, Message)
│   │   │   └── providers/
│   │   │       ├── __init__.py
│   │   │       ├── postgres.py        # PostgreSQL adapter
│   │   │       ├── mysql.py           # MySQL adapter
│   │   │       └── sqlite.py          # SQLite adapter (default local dev)
│   │   │
│   │   ├── cache/                     # Prompt & context caching
│   │   │   ├── __init__.py
│   │   │   ├── context_cache.py       # App-level knowledge context cache (invalidated on
refresh)
│   │   │   └── provider_cache.py      # Native provider caching hooks
│   │   │
│   │   ├── storage/                   # Storage abstraction
│   │   │   ├── __init__.py
│   │   │   ├── base.py                # StorageAdapter ABC
│   │   │   ├── factory.py             # Dynamic storage factory
│   │   │   └── local.py               # Local filesystem adapter
│   │   │
│   │   ├── knowledge/                 # Document loading & snapshot management
│   │   │   ├── __init__.py
│   │   │   ├── manifest.py            # Manifest validation & path safety
│   │   │   ├── loader.py              # Markdown reader & document parser
│   │   │   └── snapshot.py            # Atomic snapshot manager (safe rollback)
│   │   │
│   │   ├── strategies/                # Knowledge answering strategies
│   │   │   ├── __init__.py
│   │   │   ├── base.py                # KnowledgeStrategy ABC
│   │   │   ├── factory.py             # Dynamic strategy selector
│   │   │   ├── okf.py                 # OKF: full knowledge context in prompt
│   │   │   └── rag.py                 # RAG: retrieval + synthesis
│   │   │
│   │   ├── retrieval/                 # RAG search pipeline
│   │   │   ├── __init__.py
│   │   │   ├── chunking.py            # Prose chunker & table-aware extractor
│   │   │   ├── lexical.py             # BM25 keyword search index
│   │   │   ├── vector.py              # Dynamic vector store (Chroma/FAISS/in-memory)
│   │   │   └── hybrid.py              # Reciprocal Rank Fusion (RRF) ranker
│   │   │
│   │   ├── services/                  # Workflow orchestration
│   │   │   ├── __init__.py
│   │   │   └── chat_service.py        # Linear pipeline: guardrail → memory → strategy → LLM →
save
│   │   │
│   │   └── evaluation/                # Benchmarking
│   │       ├── __init__.py
│   │       ├── metrics.py             # Correctness, latency, attribution scoring
│   │       └── runner.py              # Batch benchmark execution
│   │
│   └── tests/
│       ├── __init__.py
│       ├── conftest.py                # Pytest fixtures, mock providers
│       ├── unit/
│       │   ├── __init__.py
│       │   ├── test_config.py
│       │   ├── test_manifest.py
│       │   ├── test_guardrail.py
│       │   ├── test_table_parsing.py
│       │   └── test_snapshot_atomic.py
│       └── integration/
│           ├── __init__.py
│           ├── test_health_api.py
│           ├── test_auth.py
│           └── test_chat_pipeline.py
```

4.1 Request Flow: Python Execution Lifecycle

When a passenger query enters the backend (`POST /api/v1/chat`), here is the exact sequence of Python calls:

```
[HTTP Request: JSON body]
       │
       ▼
1. api/routes/chat.py :: chat(request: ChatRequest)
   ├── Pydantic validates JSON schema into ChatRequest object
   └── Calls: await chat_service.process_chat(request)
       │
       ▼
2. services/chat_service.py :: process_chat()
   ├── Step A: Safety Gatekeeper
   │   └── Calls: await guardrail.classify(request.query)
   │       └── guardrail/classifier.py (Keyword fast-pass OR TinyLlama LLM check)
   │       └── If rejected: Returns polite redirection immediately (0 ms LLM cost)
   │
   ├── Step B: Multi-Turn Memory
   │   └── Calls: await memory_store.get_or_create_session(session_id)
   │   └── Calls: await memory_store.get_history(session_id)
   │       └── memory/store.py (Async SQLite fetch of last 20 turns)
   │
   ├── Step C: Knowledge Strategy Execution
   │   └── Calls: strategy = strategy_factory.get_strategy(request.strategy)
   │   └── Calls: await strategy.answer(query, history, language)
   │       ├── [OKF]: Injects complete 13-doc snapshot into system prompt
   │       └── [RAG]: Retrieves top-K table-aware BM25 chunks via HybridRetriever
   │   └── Calls: await llm_client.acomplete(messages) -> Ollama (qwen2.5:7b)
   │
   └── Step D: Persistence & Response
       ├── Calls: await memory_store.add_message(user_query)
       ├── Calls: await memory_store.add_message(assistant_response)
       └── Returns: ChatResponse object (FastAPI serializes to JSON HTTP 200)
```

---

5. Environment Variables — Complete Schema

Application & Server

Variable	Default	Description
`APP_ENV`	`development`	`development` / `production` / `test`
`APP_NAME`	`LCK Airport's AI Assistant`	Application display name
`API_V1_PREFIX`	`/api/v1`	API route prefix
`HOST`	`0.0.0.0`	Bind address inside container
`PORT`	`8000`	Backend internal port
`LOG_LEVEL`	`INFO`	`DEBUG` / `INFO` / `WARNING` / `ERROR`

Guardrail LLM (Lightweight Safety Gate)

Variable	Default	Description
`GUARDRAIL_ENABLED`	`true`	Enable/disable guardrail gate
`GUARDRAIL_MODEL`	`ollama/llama3.2:3b`	LiteLLM model string for guardrail
`GUARDRAIL_TEMPERATURE`	`0.0`	Deterministic classification
`GUARDRAIL_MAX_TOKENS`	`100`	Short classification output
`GUARDRAIL_TIMEOUT_SECONDS`	`10.0`	Fast timeout

Main LLM Provider (Dynamic)

Variable	Default	Description
`LLM_MODEL`	`ollama/llama3.1:8b`	LiteLLM model string (e.g., `ollama/llama3.1:8b`, `gpt-4o`, `claude-sonnet-4-20250514`, `gemini/gemini-2.5-flash`)
`LLM_TEMPERATURE`	`0.1`	Generation temperature
`LLM_MAX_TOKENS`	`1024`	Max output tokens
`LLM_TIMEOUT_SECONDS`	`60.0`	Request timeout

LLM Provider API Keys (Set only what you use)

Variable	Default	Description
`OLLAMA_API_BASE`	`http://host.docker.internal:11434`	Ollama server URL
`OPENAI_API_KEY`	*(none)*	OpenAI API key (`SecretStr`)
`ANTHROPIC_API_KEY`	*(none)*	Anthropic API key (`SecretStr`)
`GEMINI_API_KEY`	*(none)*	Google Gemini API key (`SecretStr`)

> **Note:** LiteLLM reads these standard env vars automatically. No custom provider factory needed.

Embedding Provider (Dynamic)

Variable	Default	Description
`EMBEDDING_MODEL`	`ollama/nomic-embed-text`	LiteLLM embedding model string

Knowledge & Storage

Variable	Default	Description
`STORAGE_PROVIDER`	`local`	`local` / `s3`
`DATA_DIR`	`/app/data`	Data mount path inside container
`MANIFEST_PATH`	`/app/data/manifest.json`	Manifest file path
`KNOWLEDGE_DIR`	`/app/data/lck_airport`	Knowledge documents directory

RAG Retrieval (Dynamic)

Variable	Default	Description
`VECTOR_STORE_PROVIDER`	`chroma`	`chroma` / `qdrant` / `faiss` / `in_memory`
`VECTOR_STORE_PATH`	`/app/data/vector_db`	Vector index storage path
`RAG_TOP_K`	`5`	Number of retrieved chunks
`RAG_CHUNK_SIZE`	`500`	Chunk size in tokens
`RAG_CHUNK_OVERLAP`	`100`	Overlap between chunks
`RAG_HYBRID_SEARCH_ENABLED`	`true`	Enable BM25 + Vector fusion

Database — Conversation Memory (Dynamic)

Variable	Default	Description
`DB_PROVIDER`	`sqlite`	`postgres` / `mysql` / `sqlite`
`DB_HOST`	`localhost`	Database host
`DB_PORT`	`5432`	Database port
`DB_NAME`	`lck_assistant`	Database name
`DB_USER`	*(none)*	Database username
`DB_PASSWORD`	*(none)*	Database password (`SecretStr`)
`DB_SQLITE_PATH`	`/app/data/lck_assistant.db`	SQLite file path

Conversation Memory

Variable	Default	Description
`MEMORY_MAX_TURNS`	`20`	Max recent turns kept verbatim

Variable	Default	Description
`MEMORY_SUMMARIZE_OLDER`	`true`	Summarize older turns via LLM
`MEMORY_SESSION_TTL_MINUTES`	`60`	Auto-expire idle sessions
`MEMORY_ENABLE_USER_LINKING`	`true`	Allow optional `user_id` linking

Prompt Caching

Variable	Default	Description
`CACHE_KNOWLEDGE_CONTEXT`	`true`	Cache assembled knowledge context in memory
`CACHE_TTL_SECONDS`	`3600`	Context cache TTL (invalidated on knowledge refresh)
`CACHE_PROVIDER_NATIVE`	`true`	Use provider-native caching when available

Security

Variable	Default	Description
`AUTH_ENABLED`	`true`	Enable authentication
`KNOWLEDGE_ADMIN_SECRET`	*(must set)*	Bearer token for protected endpoints (`SecretStr` , constant-time verified)

Streaming

Variable	Default	Description
`ENABLE_STREAMING`	`true`	Enable SSE streaming for real-time avatar TTS

---

6. API Endpoints

Method	Endpoint	Auth	Description
`GET`	`/api/v1/health`	No	System health, LLM provider connectivity, DB status
`GET`	`/api/v1/knowledge/status`	No	Active knowledge snapshot version & document count
`POST`	`/api/v1/knowledge/refresh`	**Yes (Bearer)**	Validate and activate new knowledge snapshot (atomic, rollback on failure)
`POST`	`/api/v1/chat`	No	Answer question via `okf` or `rag` strategy, optional `stream=true` , optional `session_id` and `user_id`

7. Knowledge Pipeline

Source Documents

- Approved Markdown files stored in `data/lck_airport/`
- Controlled via `data/manifest.json` (document ID, path, title, category, date, enabled flag)
- Only documents with `enabled: true` in manifest are ingested

OKF Strategy (Optimized Knowledge Feeding)

1. Load all approved Markdown documents
2. Assemble full knowledge context (cached in memory)
3. Send complete context + conversation history + question to LLM
4. LLM generates answer grounded in the provided context
5. Abstains when evidence is insufficient

RAG Strategy (Retrieval-Augmented Generation)

1. Parse Markdown into document-aware chunks (preserving tables as atomic units)
2. Build BM25 lexical index + vector embedding index
3. Hybrid search: BM25 + vector similarity with Reciprocal Rank Fusion (RRF)
4. Top-K relevant chunks sent as evidence to LLM
5. LLM generates answer with source citations
6. Abstains when retrieved evidence cannot support a reliable answer

Atomic Snapshot Management

- New knowledge snapshots are validated completely before activation
- If validation or indexing fails, the previous valid snapshot is retained
- No partially loaded or invalid snapshots can become active

---

8. Conversation Memory

Session Model

- Each conversation gets a unique `session_id` (UUID)
- Sessions can optionally be linked to a `user_id` for cross-session continuity
- Sessions auto-expire after configurable TTL (default: 60 minutes idle)

History Management

- **Sliding window:** Last 20 turns (configurable) kept verbatim in context
- **Summarization:** Turns beyond the window are summarized by the LLM into a compact context block

- **Result:** The LLM always sees a manageable context: summary of older conversation + recent turns + knowledge evidence + current question

Storage

- Dynamic database backend: PostgreSQL, MySQL, or SQLite
- Selected via `DB_PROVIDER` env var
- PostgreSQL recommended for production; SQLite for local development

---

9. Guardrail System

Every incoming user query passes through a lightweight LLM before reaching the main pipeline:

Classification	Action	Cost
`PASS`	Proceed to main pipeline	Guardrail LLM only
`OFF_TOPIC`	Return polite redirect message	Guardrail LLM only (main LLM not called)
`UNSAFE`	Block with safety message	Guardrail LLM only
`INJECTION`	Block with security response	Guardrail LLM only

- Uses the same LiteLLM integration as the main model
- Configured independently via `GUARDRAIL_*` env vars
- Can be disabled entirely with `GUARDRAIL_ENABLED=false`

---

10. Prompt Caching

Application-Level Cache

- Assembled system prompt + knowledge context is cached in memory
- Avoids rebuilding the context on every request
- Automatically invalidated when `POST /api/v1/knowledge/refresh` is called
- Configurable TTL as safety net

Provider-Native Caching

- **Anthropic Claude:** `cache_control` breakpoints on system prompt
- **OpenAI:** Automatic prefix caching for repeated long prompts
- **Gemini:** Context caching API for long-lived contexts
- **Ollama:** Benefits from KV-cache reuse with consistent prompt prefixes

---

11. Docker Compose Services

Service	Image	Port	Purpose
`nginx`	Custom (from `docker/nginx/`)	`80:80` (host)	Reverse proxy, SSL, rate limiting, SSE passthrough
`backend`	Custom (from `docker/Dockerfile`)	`8000` (internal only)	FastAPI application
`db`	`postgres:16-alpine`	`5432` (internal)	Conversation memory (optional, can use SQLite instead)

Nginx Configuration Highlights

- `proxy_buffering off` for SSE streaming
- `proxy_read_timeout 300s` for long LLM generations
- Rate limiting on `/api/v1/chat` and `/api/v1/knowledge/refresh`
- Security headers: `X-Frame-Options`, `X-Content-Type-Options`, `X-XSS-Protection`
- Internal FastAPI server headers hidden

---

12. Key Design Decisions

#	Decision	Choice
1	App name	**LCK Airport's AI Assistant**
2	Project layout	`data/` at root, `backend/` with `docker/` subdirectory
3	LLM integration	**LiteLLM** — unified interface, no framework lock-in
4	LLM providers	Dynamic via model string — Ollama, OpenAI, Claude, Gemini
5	Embeddings	Dynamic via LiteLLM embedding model string
6	Vector store	Dynamic via env — Chroma, Qdrant, FAISS, in-memory
7	Reverse proxy	**Nginx** in front of FastAPI
8	Guardrail	Lightweight LLM gate — off-topic, injection, unsafe detection
9	Conversation memory	Session-based (anonymous + optional user linking)
10	Memory storage	Dynamic DB — PostgreSQL / MySQL / SQLite via env
11	Memory management	Sliding window (20 turns) + LLM summarization of older turns
12	Prompt caching	App-level context cache + provider-native caching

#	Decision	Choice
13	Knowledge data in Git	Sample golden sets only; full datasets gitignored or in S3
14	Endpoint protection	Bearer token with `SecretStr` + constant-time comparison
15	Chat streaming	JSON response + SSE streaming (for avatar TTS)
16	No LangGraph	Linear pipeline — LangGraph is overkill for straight RAG/OKF flow

---

13. Implementation Phases

Phase	Scope
Phase 1: Skeleton	Project structure, Docker + Nginx, compose, env config, health endpoint
Phase 2: Schemas & Interfaces	Pydantic models, LiteLLM client, strategy ABC, storage ABC
Phase 3: Knowledge Pipeline	Manifest validation, Markdown loader, atomic snapshot manager
Phase 4: OKF Strategy	Full-context assembly, prompt engineering, abstention logic
Phase 5: RAG Strategy	Table-aware chunking, BM25, vector search, hybrid RRF ranking
Phase 6: Guardrail & Memory	Guardrail classifier, conversation memory with DB persistence
Phase 7: Caching & Streaming	Prompt caching, SSE streaming for avatar
Phase 8: Evaluation	Benchmark runner, OKF vs RAG comparison, metrics dashboard

---

14. Dependencies

```
[project.dependencies]
# Core
fastapi = ">=0.115"
uvicorn = {version = ">=0.30", extras = ["standard"]}
pydantic = ">=2.0"
pydantic-settings = ">=2.0"

# LLM
litellm = ">=1.40"

# Database / Memory
sqlalchemy = {version = ">=2.0", extras = ["asyncio"]}
asyncpg = ">=0.29"          # PostgreSQL async driver
aiomysql = ">=0.2"          # MySQL async driver
aiosqlite = ">=0.20"        # SQLite async driver

# RAG
rank-bm25 = ">=0.2"         # BM25 lexical search
chromadb = ">=0.5"           # Vector store (default)

# Utilities
httpx = ">=0.27"
python-dotenv = ">=1.0"

[project.optional-dependencies]
dev = [
    "pytest >= 8.0",
    "pytest-asyncio >= 0.23",
    "pytest-cov >= 5.0",
    "ruff >= 0.5",
]
```